

Documentación Técnica

Proyecto: Gestor Financiero Automatizado para Negocios

Tecnologías: Node.js, Express, PostgreSQL, Docker

Estado: Fase de Diseño

1. Introducción

El presente sistema tiene como objetivo reemplazar la gestión financiera basada en hojas de cálculo (Excel) por una aplicación web robusta y escalable. El sistema centraliza el control de ingresos diarios, la distribución automática de presupuestos basada en porcentajes y el seguimiento de deudas con proveedores.

2. Requerimientos del Sistema

2.1 Requerimientos Funcionales

- **Gestión de Saldos:** Registro manual inicial y actualización automática de saldos en Banco y Efectivo.
- **Cierre de Caja Diario:** Ingreso de ventas totales con desglose por medio de pago (Débito, Efectivo, Transferencia).
- **Distribución Automática de Fondos:** Cálculo inmediato de presupuestos según reglas configurables (Ej: 60% Mercadería, 30% Gastos Fijos, 10% Ahorro).
- **Módulo de Proveedores:** CRUD de proveedores con configuración de términos de pago (Contado / Crédito a X días).
- **Control de Egresos:** Registro de pagos vinculados a categorías específicas de presupuesto y alertas de vencimiento para facturas a crédito.
- **Dashboard de Reportes:** Visualización de salud financiera semanal, mensual y anual.

2.2 Requerimientos No Funcionales

- **Precisión Financiera:** Uso obligatorio de tipos de datos NUMERIC/DECIMAL para evitar errores de redondeo.
- **Seguridad:** Autenticación vía JWT, encriptación de contraseñas con Bcrypt y protección de cabeceras con Helmet.js.
- **Mantenibilidad:** Arquitectura de software en capas (Controller-Service-Repository).
- **Disponibilidad:** Contenerización mediante Docker para despliegue consistente.

3. Stack Tecnológico

Componente	Tecnología	Razón de elección
Entorno de Ejecución	Node.js	Alta velocidad de I/O y ecosistema robusto.
Framework Web	Express.js	Estándar de la industria, ligero y flexible.
Base de Datos	PostgreSQL	Integridad referencial y manejo profesional de transacciones.
ORM	Prisma	Facilidad para migraciones y tipado fuerte de datos.
Infraestructura	Docker / Docker Compose	Portabilidad y facilidad de levantamiento de entornos.
Seguridad	JWT + Bcrypt	Estándar para autenticación segura.

4. Arquitectura de Datos (Modelo Relacional)

Las tablas principales en PostgreSQL se definen de la siguiente manera:

1. **Accounts:** Almacena el saldo real de Banco y Efectivo.
2. **Daily_Closures:** Registro de ingresos por día.
3. **Budget_Allocation:** Desglose automático del ingreso diario en "bolsas" (Mercadería, Arriendo, Ahorro).
4. **Providers:** Catálogo de proveedores y sus condiciones de crédito.
5. **Expenses:** Registro de pagos realizados o pendientes, vinculados a un presupuesto y un proveedor.

5. Estrategia de Despliegue (Docker)

El sistema se levantará mediante tres contenedores principales definidos en docker-compose.yml:

- **api_service:** Imagen de Node.js con el backend.

- **db_service:** Imagen oficial de PostgreSQL con persistencia de datos mediante volúmenes.
- **proxy_service:** Nginx para manejo de SSL y redirección de tráfico.

Gestión de Persistencia

codeYaml

volumes:

db_data:

driver: local # Asegura que los datos sobrevivan al reinicio de contenedores

6. Plan de Desarrollo (Etapas)

1. **Etap 1: Infraestructura y Modelado:** Setup de Docker, Prisma y creación de base de datos.
2. **Etap 2: API Core:** Endpoints de proveedores y lógica de distribución automática de ingresos (60/30/10).
3. **Etap 3: Control de Egresos:** Registro de gastos y lógica de vencimientos de créditos.
4. **Etap 4: Frontend Dashboard:** Interfaz de usuario responsiva para PC y móvil.
5. **Etap 5: Seguridad y Producción:** Implementación de Login, variables de entorno y despliegue en VPS.

7. Consideraciones de Seguridad Finales

- Uso de archivos .env para credenciales.
- Backups programados de la base de datos (Postgres dump) hacia almacenamiento externo.
- Validación estricta de tipos de datos en la entrada de la API para evitar inyecciones de código.